Databases for Data Analytics

Task 4 - Aggregations and Grouping | Yaron Besser

Question 1 - Film noir / crime classical period

Task: *The classical period of film noir / crime movies was the 1940s and 1950s. (a) Create a table of classical-period movies. (b) Find all pairs of these movies that share at least 3 actors.*

(a) Building *classical_movies*

Row count: 1,898 rows

```
DROP TABLE IF EXISTS classical_movies;
CREATE TABLE classical_movies AS
    SELECT DISTINCT m.*
    FROM movies AS m
        JOIN movies_genres AS mg ON m.id = mg.movie_id
    -- Note: 'Film-Noir' is treated as a genre in the IMDB database
    WHERE mg.genre IN ('crime', 'Film-Noir')
      AND m.`year` BETWEEN 1940 AND 1959;
```

1	14:25:28	DROP TABLE IF EXISTS classical_movies	0 row(s) affected	0.0054 sec
2	14:25:28	CREATE TABLE classical_movies AS SELECT DISTINCT m.* FROM mo...	1898 row(s) affected Records: 1898 Duplicates: 0 Warnings: 0	0.305 sec

(b) Pairs with at least 3 shared actors

Row count: 6,913 pairs (intermediate film_noir_actor_movie: 42,555 rows)

```
-- Build a film-noir actor-movie table for the self-join
DROP TABLE IF EXISTS film_noir_actor_movie;
CREATE TABLE film_noir_actor_movie AS
    SELECT actor_id, movie_id
    FROM roles AS r
        JOIN classical_movies AS cm ON r.movie_id = cm.id;

SELECT fnam1.movie_id, fnam2.movie_id, COUNT(*) AS count_commen_actors
FROM film_noir_actor_movie AS fnam1
    JOIN film_noir_actor_movie AS fnam2 ON fnam1.actor_id = fnam2.actor_id
WHERE fnam1.movie_id > fnam2.movie_id
GROUP BY fnam1.movie_id, fnam2.movie_id
HAVING count_commen_actors >= 3;
```

Result and status:

movie_id	movie_id	count_commen_actors
61185	2649	5
39536	3024	3
22185	4543	3
35420	4543	3
48276	4543	3
16895	5484	3
61164	5525	6
36985	5525	4
5882	5525	3
73338	5525	3
5545	5525	3
42013	5525	3
70299	5545	3
22185	5545	3
9976	5545	3
64505	5545	3

3	14:25:28	DROP TABLE IF EXISTS film_noir_actor_movie	0 row(s) affected	0.0058 sec
4	14:25:28	CREATE TABLE film_noir_actor_movie AS SELECT actor_id, movie_id F...	42555 row(s) affected Records: 42555 Duplicates: 0 Warnings: 0	0.214 sec
5	14:25:28	SELECT fnam1.movie_id, fnam2.movie_id, COUNT(*) AS count_com...	6913 row(s) returned	0.175 sec / 0.013 sec

Explanation:

(a) Joined **movies** with **movies_genres**, filtered to genre `'crime'` or `'Film-Noir'` and years 1940-1959. `DISTINCT m.*` prevents duplicates from movies tagged with both genres. **(b)** Built **film_noir_actor_movie** as the actor-movie pairs from **classical_movies**, then joined it to itself on `actor_id`. `HAVING count_commen_actors >= 3` keeps only pairs sharing 3+ actors.

Why `fnam1.movie_id > fnam2.movie_id`:

Without this condition the self-join produces two unwanted kinds of rows: reflexive matches (same movie paired with itself) and symmetric duplicates (every real pair (A, B) also appearing as (B, A)). Strict inequality > - rather than `!=` - solves both at once: it forbids the self-pair and picks one ordering of each real pair, so each pair is counted exactly once.

Question 2 - Actors per role distribution

Task: *(a) Build a table of number of actors per role. (b) Compute the distribution. (c) Can there be roles with zero actors?*

(a) Building *role_actor_count*

Row count: 1,153,615 rows (one per distinct role)

```
DROP TABLE IF EXISTS role_actor_count;
CREATE TABLE role_actor_count AS
    SELECT `role`, COUNT(*) AS num_of_actors
    FROM roles AS r
    GROUP BY r.role;
```

(b) Distribution of actors per role

Row count: 646 buckets

```
SELECT num_of_actors, COUNT(*) AS count_role
FROM role_actor_count
GROUP BY num_of_actors
ORDER BY num_of_actors;
```

Result and status:

num_of_actors	count_role
1	985921
2	83515
3	28622
4	14078
5	8473
6	5749
7	3989
8	2976
9	2273
10	1868
11	1500
12	1283
13	1055
14	910
15	778

1	14:29:31	DROP TABLE IF EXISTS role_actor_count	0 row(s) affected	0.015 sec
2	14:29:31	CREATE TABLE role_actor_count AS SELECT `role`, COUNT(*) AS num...	1153615 row(s) affected Records: 1153615 Duplicates: 0 Warnings: 0	29.947 sec
3	14:30:01	SELECT num_of_actors, COUNT(*) AS count_role FROM role_acto...	646 row(s) returned	0.546 sec / 0.00006...

(c) Can there be roles with zero actors?

No. The **roles** table requires both a `role` string and an `actor_id` per row, so a role only exists in the data if at least one actor played it. Roles with zero actors cannot appear in this schema.

Explanation:

(a) One row per distinct `role` with the count of actor-entries playing it - 1,153,615 different role names. **(b)** Buckets the roles by their actor count: 985,921 roles have exactly 1 actor, then a long tail of higher counts (83,515 with 2 actors, 28,622 with 3, etc.). The result is sorted ascending by `num_of_actors`.

Question 3 - Typical role sets

Task: *For each movie, build a string with all its role names. Use only roles played by at least 10 actors; deduplicate; order doesn't matter (use sorted form).*

Row count: 178,254 movies

```
SELECT m.id, m.name,
    GROUP_CONCAT(DISTINCT r2.role ORDER BY r2.role) AS role_str
FROM (
    SELECT role
    FROM roles
    WHERE role != ''
    GROUP BY role
    HAVING COUNT(*) >= 10
) AS r
JOIN roles  AS r2 ON r.role = r2.role
JOIN movies AS m  ON r2.movie_id = m.id
GROUP BY m.id, m.name;
```

Result and status:

id	name	role_str
2	$	Attorney,Berta,Bodyguard,Bruno,Helga,Joe Collins,Karl,Major,Sarge
6	$1,000,000 Duck	Assayer,Bank Manager,Bank Teller,Car Salesman,Morgan,Mr. Smith,Parking Attendant
11	$1000 a Touchdown	Babe,Bertie,Big Boy,Billie,Blondie,Buck,Cab Driver,Cecil,Coach,Dick,Dora,Dr. Black,Duke,Gen,Ginger,Guard,Hank,Harry,Henry,Honey,Hysterical Wom
15	$30	Dad,Scott
18	$40,000	Bike Messenger,Butler,Chauffeur,Dean,Dick,Dirk,Dog,Frenchman,Gray,Jackie,Maitre'd,Operator,Party Guest,Receptionist,Rene,Secret Service Man,Se
19	$5,000 Reward	Aunt Kate,Jeanette,Tracy
20	$5,000,000 Counterfeiting Plot, The	Himself,John Walton,Mary Kelly
21	$5.15/Hr.	Brandy,Customer #1,Customer #2,Joy,Trucker
22	$5.20 an Hour Dream, The	Billy Boy,Checker,Ginny,Randy,Rawlings
23	$50,000 Challenge, The	Announcer,Host
26	$50,000 Reward	Follies girl,Freckles,Mrs. Miller,Snowball
31	$pent	Detective,Doug,Evan,Female Cop,Francine,Grant,Jay,Jessica,Kimberly,Male Cop,Max,Mr. Walsh,Mrs. Walsh,Nathan,Ron,Scott,Steve,Waitress
32	$ucces Part One	Herself,Himself
33	$windle	Barfly,Bartender,Cameraman,Cisco,Eddie,Gambler,Hooker,Judy,Junior,Lawrence,Lukas,Man In Crowd,Marc,Reporter,Waitress
36	'15'	Himself

1 14:30:46 SELECT m.id, m.name, GROUP_CONCAT(DISTINCT r2.role... 178254 row(s) returned 41.814 sec / 0.735 sec

Explanation:

The inner subquery picks roles that appear at least 10 times in the **roles** table, excluding empty role names. Joining this back to **roles** and **movies** filters each movie's cast down to those frequent roles only. `GROUP_CONCAT(DISTINCT r2.role ORDER BY r2.role)` builds one string per movie containing all its frequent role names, deduplicated and sorted alphabetically. Two movies with the same cast structure produce the same string, which makes them easy to compare.

Question 4 - Role probability per movie and per genre

Task: *Compute the probability of each role to appear in a movie and in a genre (e.g., the share of comedies that contain a 'clown' role).*

Row count: 1,742,524 rows (role × genre combinations); *genere_count*: 21 rows

```
-- Build a per-genre count table
DROP TABLE IF EXISTS genere_count;
CREATE TABLE genere_count AS
    SELECT genre, COUNT(*) AS count_genre
    FROM movies_genres
    GROUP BY genre;

SELECT
    r.role,
    COUNT(*) / (SELECT COUNT(*) FROM movies) AS prop_of_movies,
    mg.genre,
    COUNT(*) / gc.count_genre AS prop_of_genre
FROM roles AS r
    JOIN movies_genres AS mg ON r.movie_id = mg.movie_id
    JOIN genere_count  AS gc ON mg.genre = gc.genre
WHERE r.role != ''
GROUP BY r.role, mg.genre, gc.count_genre;
```

Result and status:

role	prop_of_movies	genre	prop_of_genre
(1991 reissue only)	0.0000	Adult	0.0001
(segment "A Boca")	0.0000	Comedy	0.0000
(segment "A Suspeita")	0.0000	Comedy	0.0000
(segment "Head Like a Hole")	0.0000	Documentary	0.0000
(segment "Head Like a Hole")	0.0000	Music	0.0002
(segment "La voce umana")	0.0000	Drama	0.0000
(segment "O Apartamento")	0.0000	Comedy	0.0000
(segment "O Filminho")	0.0000	Comedy	0.0000
(segment "O Torneio")	0.0000	Comedy	0.0000
"Abe Lincoln"	0.0000	Mystery	0.0002
"Abe Lincoln"	0.0000	Thriller	0.0001
"American Eagle"	0.0000	Mystery	0.0002
"American Eagle"	0.0000	Thriller	0.0001
"Astoria" Owner	0.0000	Drama	0.0000
"Astoria" Owner	0.0000	War	0.0002
"Bar From Hell" Patron	0.0000	Horror	0.0001
"Betsy Ross"	0.0000	Mystery	0.0002
"Betsy Ross"	0.0000	Thriller	0.0001
"Betty Bumcakes" Actor	0.0000	Drama	0.0000
"Betty Bumcakes" Actor	0.0000	Horror	0.0001

	Time	Action	Response	Duration / Fetch Time
1	14:32:08	DROP TABLE IF EXISTS genere_count	0 row(s) affected	0.011 sec
2	14:32:08	CREATE TABLE genere_count AS SELECT genre, COUNT(*) AS count_...	21 row(s) affected Records: 21 Duplicates: 0 Warnings: 0	0.364 sec
3	14:32:08	SELECT r.role, count(*)/(select count(*) from movies) AS prop_of_...	1742524 row(s) returned	39.762 sec / 1.911 sec

Explanation:

First built **genere_count** with the total number of movies in each genre - 21 rows, one per genre. Then joined **roles** with **movies_genres** and **genere_count**, grouped by (role, genre), and computed two ratios. `prop_of_movies = COUNT(*) / total movies` gives the share of all movies that contain this role-genre pairing. `prop_of_genre = COUNT(*) / count_genre` gives the share of movies *within that genre* that contain this role. Filtered out empty role names.

Question 5 - Interesting question: actor-genre concentration

Question: *For each actor, what fraction of their movies falls in each genre? This shows where each actor is concentrated in their career.*

Row count: 1,893,605 rows (full result, 59.173 sec)

```
SELECT
    ac.id AS actor_id,
    ac.first_name, ac.last_name,
    mg.genre,
    COUNT(DISTINCT r2.movie_id) / a_movies_count AS prop_genre
FROM (
    SELECT a.id, a.first_name, a.last_name,
           COUNT(DISTINCT r.movie_id) AS a_movies_count
    FROM actors AS a
        JOIN roles AS r ON a.id = r.actor_id
    GROUP BY a.id
) AS ac
JOIN roles AS r2 ON ac.id = r2.actor_id
JOIN movies_genres AS mg ON r2.movie_id = mg.movie_id
GROUP BY ac.id, ac.first_name, ac.last_name, mg.genre
ORDER BY actor_id, prop_genre DESC;
```

Result and status:

actor_id	first_name	last_name	genre	prop_genre
93	Kim Fupz	Aakeson	Romance	0.3333
93	Kim Fupz	Aakeson	Short	0.3333
94	Steve	Aalam	Action	0.2857
94	Steve	Aalam	Crime	0.2857
94	Steve	Aalam	Drama	0.2857
94	Steve	Aalam	Romance	0.2857
94	Steve	Aalam	Short	0.2857
94	Steve	Aalam	Thriller	0.2857
94	Steve	Aalam	Comedy	0.1429
94	Steve	Aalam	Documentary	0.1429
94	Steve	Aalam	Sci-Fi	0.1429
94	Steve	Aalam	War	0.1429
95	Freddy	Aalbeck	Drama	1.0000
95	Freddy	Aalbeck	Musical	1.0000
96	George	Aalbregise	Thriller	1.0000
97	Niels	Aalbæk Jensen	Thriller	1.0000
98	Karsten	Aalholm	Music	1.0000
99	Preben	Aalholt	Drama	1.0000
99	Preben	Aalholt	Action	0.5000
99	Preben	Aalholt	Documentary	0.5000
99	Preben	Aalholt	War	0.5000
100	Henri	Aalto	Drama	1.0000
100	Henri	Aalto	Family	1.0000

1 15:22:04 SELECT ac.id AS actor_id, ac.first_name, ac.last_name, mg.genre, COUNT(DISTINCT... 1893605 row(s) returned 59.173 sec / 1.038 sec

Explanation:

Inner subquery **ac** computes each actor's total distinct movies (`a_movies_count`). Joining this back to **roles** and **movies_genres**, then grouping by (actor, genre), gives the share of each actor's movies that fall in each genre: `prop_genre = COUNT(DISTINCT r2.movie_id) / a_movies_count`.

Why `COUNT(DISTINCT movie_id)`:

An actor can have multiple **roles** entries for the same movie, and a movie can have multiple genre tags. Without `DISTINCT`, a movie tagged with two genres would be double-counted for each role entry the actor has in it. `DISTINCT movie_id` guarantees we count *movies*, not row pairings - both in the inner subquery (total movies) and in the outer count (movies per genre).

Why probabilities can sum to over 1 for one actor:

Movies in the IMDB schema can carry multiple genre tags. A film tagged both Comedy and Romance contributes to *both* genre proportions for its actor. Example from the result: Steve Aalam (id 94) has 7 distinct movies, and his `prop_genre` values sum to ~2.29 across his genres - meaning his films average about 2.3 genre tags each. `prop_genre` measures *presence* in each genre, not exclusive concentration.